Міністерство освіти і науки України
НТУУ «Київський політехнічний інститут»
Навчально-науковий Фізико-технічний інститут

Лабораторна робота
Дослідження універсальності представлень SSM

Виконали студентки
4 курсу НН ФТІ, групи ФІ-21
Максакова Софія та Мерзла Ірина

Київ 2026

Мета: Дослідити універсальність та доменно-незалежні властивості простору ознак моделі станів (SSM), попередньо навченої на текстових даних, шляхом застосування методу лінійного зондування (linear probing) для задач класифікації зображень без перенавчання базової моделі.

Методика дослідження: У роботі використовується попередньо навчена модель станів (SSM), натренована на текстових даних. Параметри базової моделі залишаються незмінними (замороженими). До моделі додається новий лінійний класифікаційний шар, який навчається на даних іншої природи — зображеннях, представлених у вигляді послідовностей пікселів або патчів. На вхід моделі подаються перетворені зображення (наприклад, з наборів MNIST або CIFAR), які подаються як послідовності. Навчання виконується лише для класифікаційного шару без оновлення ваг базової SSM. Для порівняння використовується також випадково ініціалізована модель SSM із замороженими параметрами.

Критерії оцінювання та метрики: Оцінювання ефективності моделі здійснюється за допомогою точності класифікації (accuracy) на тестовій вибірці. Аналізується швидкість збіжності моделі при навчанні як на обмеженій кількості даних (few-shot), так і на повному наборі даних. Результати порівнюються між моделлю з попередньо навченими вагами та випадково ініціалізованою моделлю для визначення впливу попереднього навчання на якість представлень.

Виконання лабораторної роботи

Метою даного проєкту є дослідження можливості використання State Space Models (SSM), зокрема моделі Mamba, для задачі класифікації зображень. Основна ідея полягає у застосуванні моделей, що працюють із послідовностями, до обробки зображень шляхом перетворення їх у відповідний послідовнісний формат.

Розподіл роботи між учасницями

У межах проєкту обов'язки були поділені між двома учасницями. Мерзла Ірина (Person 1) відповідала за підготовку даних та модельну частину. Вона реалізувала конфігурацію проєкту, підготовку датасету та перетворення зображень у послідовність, необхідну для роботи Mamba. Також нею було побудовано сам backbone на основі mamba-ssm, створено варіанти моделі із замороженим і повністю навчуваним backbone (основну частину моделі, що формує представлення даних), а також виконано первинну перевірку коректності роботи всієї моделі.

Максакова Софія (Person 2) відповідала за навчання моделей та їх оцінювання. З її боку було реалізовано процес навчання, обчислення метрик і сценарії запуску експериментів. Вона провела навчання моделі із замороженим backbone, експеримент із повним донавчанням Mamba, а також реалізувала базову CNN-модель для порівняння. Крім цього, було виконано аналіз отриманих результатів і сформульовано підсумкові висновки щодо ефективності кожного підходу.

Послідовність виконання роботи

На початку було організовано структуру проєкту та реалізовано базову обробку даних, яка включала перетворення зображень у послідовність. Далі було побудовано backbone на основі Mamba і створено два варіанти моделі: із замороженим backbone та з можливістю його навчання.

Після цього було реалізовано процес навчання моделей і проведено експерименти. Спочатку перевірено коректність роботи всієї архітектури, далі виконано навчання моделі із замороженим backbone, потім — навчання моделі з повним донавчанням, і на завершення — навчання базової CNN-моделі. Отримані результати були проаналізовані для подальших висновків.

Структура проєкту

```
├── README.md
├── configs
│   └── config.py
├── data
│   ├── cifar-10-batches-py
│   │   ├── batches.meta
│   │   ├── data_batch_1
│   │   ├── data_batch_2
│   │   ├── data_batch_3
│   │   ├── data_batch_4
│   │   ├── data_batch_5
│   │   ├── readme.html
│   │   └── test_batch
│   ├── cifar-10-python.tar.gz
│   └── dataset.py
├── experiments
│   ├── run_baseline.py
│   ├── run_finetune.py
│   ├── run_frozen.py
│   └── run_frozen_train.py
├── models
│   ├── baseline_cnn.py
│   ├── finetune_model.py
│   ├── frozen_model.py
│   └── mamba_backbone.py
└── training
    ├── evaluate.py
    └── train.py
```

Проєкт має модульну структуру. У папці configs зберігаються параметри запуску, у data реалізовано підготовку даних, у models знаходяться всі моделі, включаючи backbone (основну частину моделі, що відповідає за витяг ознак), frozen і finetune варіанти та CNN baseline. Папка training містить логіку навчання і оцінювання, а experiments — сценарії запуску різних експериментів. Така структура дозволяє чітко розділити функціональність і спростити роботу з проєктом.

Результати

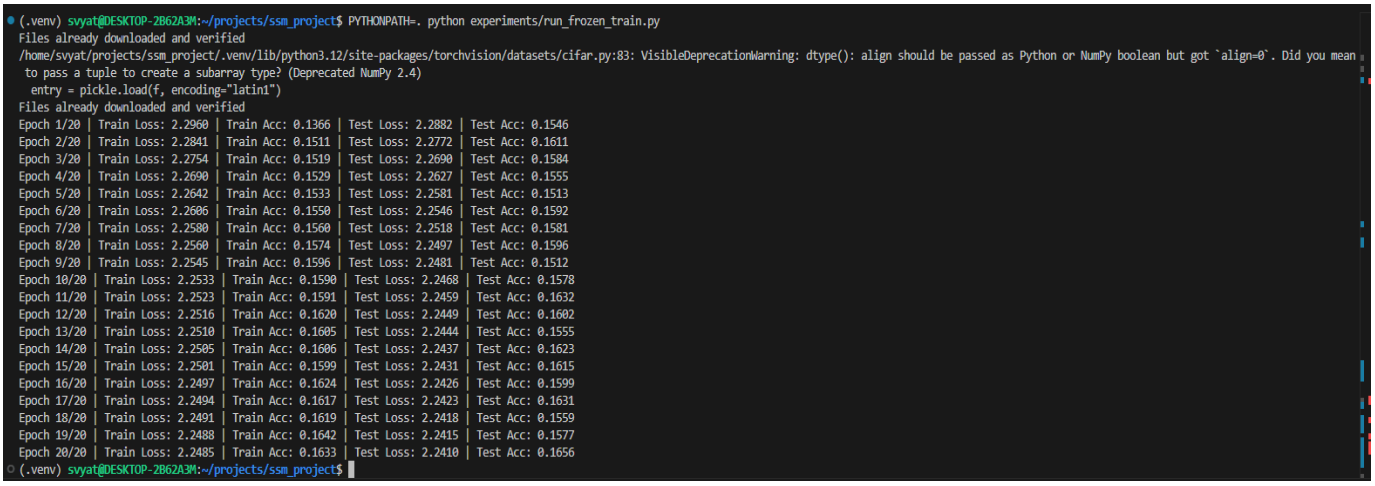
У ході роботи було досліджено три підходи: використання замороженого backbone, донавчання всієї моделі та застосування класичної згорткової нейронної мережі. Кожен із цих підходів показав різний рівень ефективності, що дозволяє оцінити вплив способу навчання моделі на кінцевий результат.

Загалом встановлено, що модель із замороженим backbone демонструє низьку точність і практично не покращує результати в процесі навчання. У свою чергу, донавчання моделі дозволяє суттєво підвищити якість класифікації, що свідчить про здатність Mamba адаптуватися до нової задачі. При цьому базова CNN-модель показує найкращий результат серед усіх розглянутих варіантів.

Детальні результати навчання моделей, включаючи значення точності та функції втрат по епохах, наведено нижче у вигляді скріншотів виконання експериментів.

1. Frozen Mamba

Для перевірки ефективності використання Mamba як backbone (основної частини моделі, що витягує ознаки) було проведено експеримент із замороженими параметрами backbone. У цьому випадку навчався лише класифікатор. Результати навчання наведені на рисунку 1.



```
(.venv) svyat@DESKTOP-2B62A3M:~/projects/ssm_project$ PYTHONPATH=. python experiments/run_frozen_train.py
Files already downloaded and verified
/home/svyat/projects/ssm_project/.venv/lib/python3.12/site-packages/torchvision/datasets/cifar.py:83: VisibleDeprecationWarning: dtype(): align should be passed as Python or NumPy boolean but got `align=0`. Did you mean
 to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
  entry = pickle.load(f, encoding="latin1")
Files already downloaded and verified
Epoch 1/20 | Train Loss: 2.2960 | Train Acc: 0.1366 | Test Loss: 2.2882 | Test Acc: 0.1546
Epoch 2/20 | Train Loss: 2.2841 | Train Acc: 0.1511 | Test Loss: 2.2772 | Test Acc: 0.1611
Epoch 3/20 | Train Loss: 2.2754 | Train Acc: 0.1519 | Test Loss: 2.2690 | Test Acc: 0.1584
Epoch 4/20 | Train Loss: 2.2690 | Train Acc: 0.1529 | Test Loss: 2.2627 | Test Acc: 0.1555
Epoch 5/20 | Train Loss: 2.2642 | Train Acc: 0.1533 | Test Loss: 2.2581 | Test Acc: 0.1513
Epoch 6/20 | Train Loss: 2.2606 | Train Acc: 0.1550 | Test Loss: 2.2546 | Test Acc: 0.1592
Epoch 7/20 | Train Loss: 2.2580 | Train Acc: 0.1560 | Test Loss: 2.2518 | Test Acc: 0.1581
Epoch 8/20 | Train Loss: 2.2560 | Train Acc: 0.1574 | Test Loss: 2.2497 | Test Acc: 0.1596
Epoch 9/20 | Train Loss: 2.2545 | Train Acc: 0.1596 | Test Loss: 2.2481 | Test Acc: 0.1512
Epoch 10/20 | Train Loss: 2.2533 | Train Acc: 0.1590 | Test Loss: 2.2468 | Test Acc: 0.1578
Epoch 11/20 | Train Loss: 2.2523 | Train Acc: 0.1591 | Test Loss: 2.2459 | Test Acc: 0.1632
Epoch 12/20 | Train Loss: 2.2516 | Train Acc: 0.1620 | Test Loss: 2.2449 | Test Acc: 0.1602
Epoch 13/20 | Train Loss: 2.2510 | Train Acc: 0.1605 | Test Loss: 2.2444 | Test Acc: 0.1555
Epoch 14/20 | Train Loss: 2.2505 | Train Acc: 0.1606 | Test Loss: 2.2437 | Test Acc: 0.1623
Epoch 15/20 | Train Loss: 2.2501 | Train Acc: 0.1599 | Test Loss: 2.2431 | Test Acc: 0.1615
Epoch 16/20 | Train Loss: 2.2497 | Train Acc: 0.1624 | Test Loss: 2.2426 | Test Acc: 0.1599
Epoch 17/20 | Train Loss: 2.2494 | Train Acc: 0.1617 | Test Loss: 2.2423 | Test Acc: 0.1631
Epoch 18/20 | Train Loss: 2.2491 | Train Acc: 0.1619 | Test Loss: 2.2418 | Test Acc: 0.1559
Epoch 19/20 | Train Loss: 2.2488 | Train Acc: 0.1642 | Test Loss: 2.2415 | Test Acc: 0.1577
Epoch 20/20 | Train Loss: 2.2485 | Train Acc: 0.1633 | Test Loss: 2.2410 | Test Acc: 0.1656
(.venv) svyat@DESKTOP-2B62A3M:~/projects/ssm_project$
```

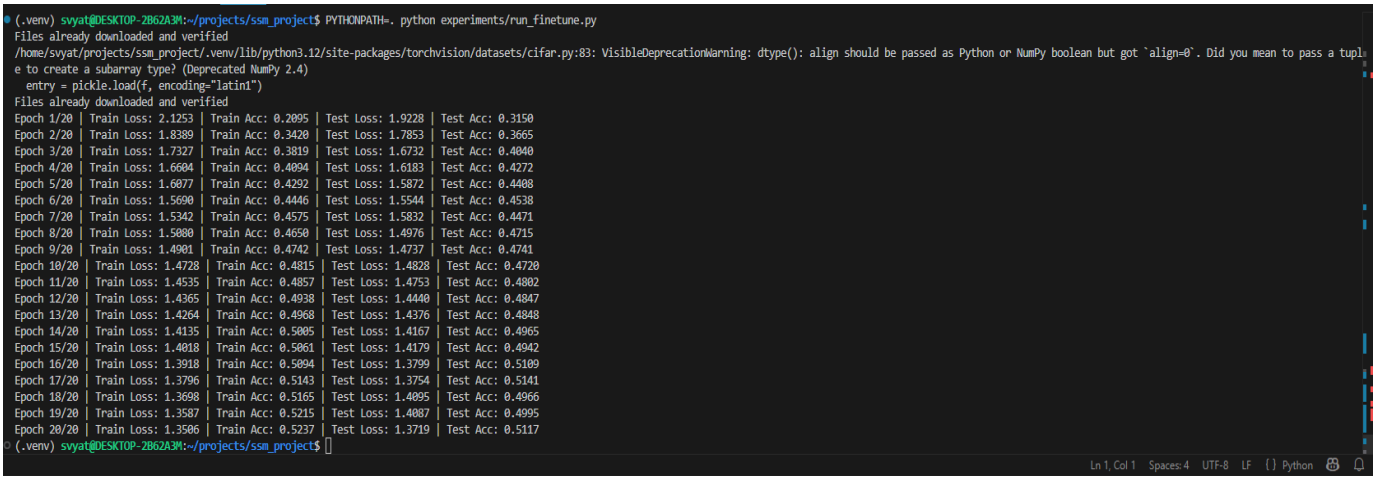
Рисунок 1 – Результати навчання моделі із замороженим backbone

```
Epoch 1/20 | Train Loss: 2.2960 | Train Acc: 0.1366 | Test Loss: 2.2882 | Test Acc: 0.1546
Epoch 2/20 | Train Loss: 2.2841 | Train Acc: 0.1511 | Test Loss: 2.2772 | Test Acc: 0.1611
Epoch 3/20 | Train Loss: 2.2754 | Train Acc: 0.1519 | Test Loss: 2.2690 | Test Acc: 0.1584
Epoch 4/20 | Train Loss: 2.2690 | Train Acc: 0.1529 | Test Loss: 2.2627 | Test Acc: 0.1555
Epoch 5/20 | Train Loss: 2.2642 | Train Acc: 0.1533 | Test Loss: 2.2581 | Test Acc: 0.1513
Epoch 6/20 | Train Loss: 2.2606 | Train Acc: 0.1550 | Test Loss: 2.2546 | Test Acc: 0.1592
Epoch 7/20 | Train Loss: 2.2580 | Train Acc: 0.1560 | Test Loss: 2.2518 | Test Acc: 0.1581
Epoch 8/20 | Train Loss: 2.2560 | Train Acc: 0.1574 | Test Loss: 2.2497 | Test Acc: 0.1596
Epoch 9/20 | Train Loss: 2.2545 | Train Acc: 0.1596 | Test Loss: 2.2481 | Test Acc: 0.1512
Epoch 10/20 | Train Loss: 2.2533 | Train Acc: 0.1590 | Test Loss: 2.2468 | Test Acc: 0.1578
Epoch 11/20 | Train Loss: 2.2523 | Train Acc: 0.1591 | Test Loss: 2.2459 | Test Acc: 0.1632
Epoch 12/20 | Train Loss: 2.2516 | Train Acc: 0.1620 | Test Loss: 2.2449 | Test Acc: 0.1602
Epoch 13/20 | Train Loss: 2.2510 | Train Acc: 0.1605 | Test Loss: 2.2444 | Test Acc: 0.1555
Epoch 14/20 | Train Loss: 2.2505 | Train Acc: 0.1606 | Test Loss: 2.2437 | Test Acc: 0.1623
Epoch 15/20 | Train Loss: 2.2501 | Train Acc: 0.1599 | Test Loss: 2.2431 | Test Acc: 0.1615
Epoch 16/20 | Train Loss: 2.2497 | Train Acc: 0.1624 | Test Loss: 2.2426 | Test Acc: 0.1599
Epoch 17/20 | Train Loss: 2.2494 | Train Acc: 0.1617 | Test Loss: 2.2423 | Test Acc: 0.1631
Epoch 18/20 | Train Loss: 2.2491 | Train Acc: 0.1619 | Test Loss: 2.2418 | Test Acc: 0.1559
Epoch 19/20 | Train Loss: 2.2488 | Train Acc: 0.1642 | Test Loss: 2.2415 | Test Acc: 0.1577
Epoch 20/20 | Train Loss: 2.2485 | Train Acc: 0.1633 | Test Loss: 2.2410 | Test Acc: 0.1656
```

Як видно з результатів, точність моделі залишається на рівні приблизно 16% і майже не зростає з кількістю епох. Це свідчить про те, що заморожений backbone не може ефективно виконувати роль екстрактора ознак для даної задачі.

2. Finetuned Mamba

Наступним кроком було проведено експеримент із повним донавчанням моделі, де backbone (основна частина моделі, що витягує ознаки) навчався разом із класифікатором. Результати навчання наведені на рисунку 2.



```
(.venv) svyat@DESKTOP-2B62A3M:~/projects/ssm_project$ PYTHONPATH=. python experiments/run_finetune.py
Files already downloaded and verified
/home/svyat/projects/ssm_project/.venv/lib/python3.12/site-packages/torchvision/datasets/cifar.py:83: VisibleDeprecationWarning: dtype(): align should be passed as Python or NumPy boolean but got `align=0`. Did you mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
  entry = pickle.load(f, encoding="latin1")
Files already downloaded and verified
Epoch 1/20 | Train Loss: 2.1253 | Train Acc: 0.2095 | Test Loss: 1.9228 | Test Acc: 0.3150
Epoch 2/20 | Train Loss: 1.8389 | Train Acc: 0.3420 | Test Loss: 1.7853 | Test Acc: 0.3665
Epoch 3/20 | Train Loss: 1.7327 | Train Acc: 0.3819 | Test Loss: 1.6732 | Test Acc: 0.4040
Epoch 4/20 | Train Loss: 1.6604 | Train Acc: 0.4094 | Test Loss: 1.6183 | Test Acc: 0.4272
Epoch 5/20 | Train Loss: 1.6077 | Train Acc: 0.4292 | Test Loss: 1.5872 | Test Acc: 0.4408
Epoch 6/20 | Train Loss: 1.5690 | Train Acc: 0.4446 | Test Loss: 1.5544 | Test Acc: 0.4538
Epoch 7/20 | Train Loss: 1.5342 | Train Acc: 0.4575 | Test Loss: 1.5832 | Test Acc: 0.4471
Epoch 8/20 | Train Loss: 1.5080 | Train Acc: 0.4650 | Test Loss: 1.4976 | Test Acc: 0.4715
Epoch 9/20 | Train Loss: 1.4901 | Train Acc: 0.4742 | Test Loss: 1.4737 | Test Acc: 0.4741
Epoch 10/20 | Train Loss: 1.4728 | Train Acc: 0.4815 | Test Loss: 1.4828 | Test Acc: 0.4720
Epoch 11/20 | Train Loss: 1.4535 | Train Acc: 0.4857 | Test Loss: 1.4753 | Test Acc: 0.4802
Epoch 12/20 | Train Loss: 1.4365 | Train Acc: 0.4938 | Test Loss: 1.4440 | Test Acc: 0.4847
Epoch 13/20 | Train Loss: 1.4264 | Train Acc: 0.4968 | Test Loss: 1.4376 | Test Acc: 0.4848
Epoch 14/20 | Train Loss: 1.4135 | Train Acc: 0.5005 | Test Loss: 1.4167 | Test Acc: 0.4965
Epoch 15/20 | Train Loss: 1.4018 | Train Acc: 0.5061 | Test Loss: 1.4179 | Test Acc: 0.4942
Epoch 16/20 | Train Loss: 1.3918 | Train Acc: 0.5094 | Test Loss: 1.3799 | Test Acc: 0.5109
Epoch 17/20 | Train Loss: 1.3796 | Train Acc: 0.5143 | Test Loss: 1.3754 | Test Acc: 0.5141
Epoch 18/20 | Train Loss: 1.3698 | Train Acc: 0.5165 | Test Loss: 1.4095 | Test Acc: 0.4966
Epoch 19/20 | Train Loss: 1.3587 | Train Acc: 0.5215 | Test Loss: 1.4087 | Test Acc: 0.4995
Epoch 20/20 | Train Loss: 1.3506 | Train Acc: 0.5237 | Test Loss: 1.3719 | Test Acc: 0.5117
(.venv) svyat@DESKTOP-2B62A3M:~/projects/ssm_project$
```

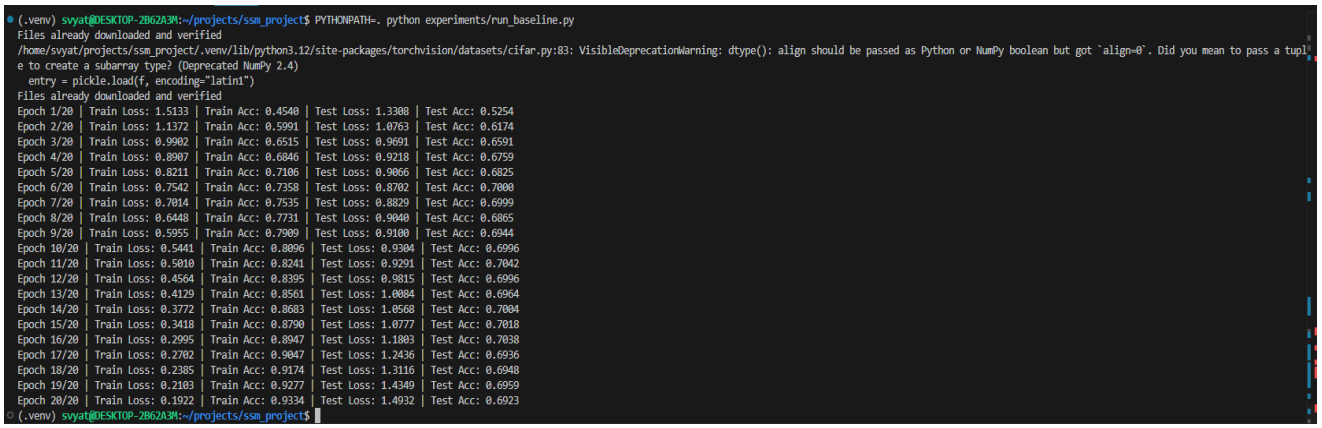
Рисунок 2 – Результати навчання моделі з донавчанням backbone

```
Epoch 1/20 | Train Loss: 2.1253 | Train Acc: 0.2095 | Test Loss: 1.9228 | Test Acc: 0.3150
Epoch 2/20 | Train Loss: 1.8389 | Train Acc: 0.3420 | Test Loss: 1.7853 | Test Acc: 0.3665
Epoch 3/20 | Train Loss: 1.7327 | Train Acc: 0.3819 | Test Loss: 1.6732 | Test Acc: 0.4040
Epoch 4/20 | Train Loss: 1.6604 | Train Acc: 0.4094 | Test Loss: 1.6183 | Test Acc: 0.4272
Epoch 5/20 | Train Loss: 1.6077 | Train Acc: 0.4292 | Test Loss: 1.5872 | Test Acc: 0.4408
Epoch 6/20 | Train Loss: 1.5690 | Train Acc: 0.4446 | Test Loss: 1.5544 | Test Acc: 0.4538
Epoch 7/20 | Train Loss: 1.5342 | Train Acc: 0.4575 | Test Loss: 1.5832 | Test Acc: 0.4471
Epoch 8/20 | Train Loss: 1.5080 | Train Acc: 0.4650 | Test Loss: 1.4976 | Test Acc: 0.4715
Epoch 9/20 | Train Loss: 1.4901 | Train Acc: 0.4742 | Test Loss: 1.4737 | Test Acc: 0.4741
Epoch 10/20 | Train Loss: 1.4728 | Train Acc: 0.4815 | Test Loss: 1.4828 | Test Acc: 0.4720
Epoch 11/20 | Train Loss: 1.4535 | Train Acc: 0.4800 | Test Loss: 1.4753 | Test Acc: 0.4802
Epoch 12/20 | Train Loss: 1.4365 | Train Acc: 0.4938 | Test Loss: 1.4440 | Test Acc: 0.4847
Epoch 13/20 | Train Loss: 1.4264 | Train Acc: 0.4968 | Test Loss: 1.4376 | Test Acc: 0.4848
Epoch 14/20 | Train Loss: 1.4135 | Train Acc: 0.5005 | Test Loss: 1.4167 | Test Acc: 0.4965
Epoch 15/20 | Train Loss: 1.4018 | Train Acc: 0.5061 | Test Loss: 1.4179 | Test Acc: 0.4942
Epoch 16/20 | Train Loss: 1.3918 | Train Acc: 0.5094 | Test Loss: 1.3799 | Test Acc: 0.5109
Epoch 17/20 | Train Loss: 1.3796 | Train Acc: 0.5143 | Test Loss: 1.3754 | Test Acc: 0.5141
Epoch 18/20 | Train Loss: 1.3698 | Train Acc: 0.5165 | Test Loss: 1.4095 | Test Acc: 0.4966
Epoch 19/20 | Train Loss: 1.3587 | Train Acc: 0.5215 | Test Loss: 1.4087 | Test Acc: 0.4995
Epoch 20/20 | Train Loss: 1.3506 | Train Acc: 0.5237 | Test Loss: 1.3719 | Test Acc: 0.5117
```

У цьому випадку спостерігається суттєве зростання точності, яка досягає приблизно 51%. Це означає, що модель здатна адаптувати свої внутрішні представлення до задачі класифікації.

3. CNN Baseline

Для порівняння було реалізовано базову згорткову нейронну мережу. Вона не використовує Mamba, але добре підходить для задач класифікації зображень. Результати навчання наведені на рисунку 3.



```
(.venv) svyat@DESKTOP-2B62A3M:~/projects/ssm_project$ PYTHONPATH=. python experiments/run_baseline.py
Files already downloaded and verified
/home/svyat/projects/ssm_project/.venv/lib/python3.12/site-packages/torchvision/datasets/cifar.py:83: VisibleDeprecationWarning: dtype(): align should be passed as Python or NumPy boolean but got `align=0`. Did you mean to pass a tupl
e to create a subarray type? (Deprecated NumPy 2.4)
  entry = pickle.load(f, encoding="latin1")
Files already downloaded and verified
Epoch 1/20 | Train Loss: 1.5133 | Train Acc: 0.4540 | Test Loss: 1.3308 | Test Acc: 0.5254
Epoch 2/20 | Train Loss: 1.1372 | Train Acc: 0.5991 | Test Loss: 1.0763 | Test Acc: 0.6174
Epoch 3/20 | Train Loss: 0.9902 | Train Acc: 0.6515 | Test Loss: 0.9691 | Test Acc: 0.6591
Epoch 4/20 | Train Loss: 0.8907 | Train Acc: 0.6846 | Test Loss: 0.9218 | Test Acc: 0.6759
Epoch 5/20 | Train Loss: 0.8211 | Train Acc: 0.7106 | Test Loss: 0.9066 | Test Acc: 0.6825
Epoch 6/20 | Train Loss: 0.7542 | Train Acc: 0.7358 | Test Loss: 0.8702 | Test Acc: 0.7000
Epoch 7/20 | Train Loss: 0.7014 | Train Acc: 0.7535 | Test Loss: 0.8829 | Test Acc: 0.6999
Epoch 8/20 | Train Loss: 0.6448 | Train Acc: 0.7731 | Test Loss: 0.9040 | Test Acc: 0.6865
Epoch 9/20 | Train Loss: 0.5955 | Train Acc: 0.7909 | Test Loss: 0.9100 | Test Acc: 0.6944
Epoch 10/20 | Train Loss: 0.5441 | Train Acc: 0.8096 | Test Loss: 0.9304 | Test Acc: 0.6996
Epoch 11/20 | Train Loss: 0.5010 | Train Acc: 0.8241 | Test Loss: 0.9291 | Test Acc: 0.7042
Epoch 12/20 | Train Loss: 0.4564 | Train Acc: 0.8395 | Test Loss: 0.9815 | Test Acc: 0.6996
Epoch 13/20 | Train Loss: 0.4129 | Train Acc: 0.8561 | Test Loss: 1.0084 | Test Acc: 0.6964
Epoch 14/20 | Train Loss: 0.3772 | Train Acc: 0.8683 | Test Loss: 1.0568 | Test Acc: 0.7004
Epoch 15/20 | Train Loss: 0.3418 | Train Acc: 0.8790 | Test Loss: 1.0777 | Test Acc: 0.7018
Epoch 16/20 | Train Loss: 0.2995 | Train Acc: 0.8947 | Test Loss: 1.1803 | Test Acc: 0.7038
Epoch 17/20 | Train Loss: 0.2702 | Train Acc: 0.9047 | Test Loss: 1.2436 | Test Acc: 0.6936
Epoch 18/20 | Train Loss: 0.2385 | Train Acc: 0.9174 | Test Loss: 1.3116 | Test Acc: 0.6948
Epoch 19/20 | Train Loss: 0.2103 | Train Acc: 0.9277 | Test Loss: 1.4349 | Test Acc: 0.6959
Epoch 20/20 | Train Loss: 0.1922 | Train Acc: 0.9334 | Test Loss: 1.4932 | Test Acc: 0.6923
(.venv) svyat@DESKTOP-2B62A3M:~/projects/ssm_project$
```

Рисунок 3 – Результати навчання CNN моделі

```
Epoch 1/20 | Train Loss: 1.5133 | Train Acc: 0.4540 | Test Loss: 1.3308 | Test Acc: 0.5254
Epoch 2/20 | Train Loss: 1.1372 | Train Acc: 0.5991 | Test Loss: 1.0763 | Test Acc: 0.6174
Epoch 3/20 | Train Loss: 0.9902 | Train Acc: 0.6515 | Test Loss: 0.9691 | Test Acc: 0.6591
Epoch 4/20 | Train Loss: 0.8907 | Train Acc: 0.6846 | Test Loss: 0.9218 | Test Acc: 0.6759
Epoch 5/20 | Train Loss: 0.8211 | Train Acc: 0.7106 | Test Loss: 0.9066 | Test Acc: 0.6825
Epoch 6/20 | Train Loss: 0.7542 | Train Acc: 0.7358 | Test Loss: 0.8702 | Test Acc: 0.7000
Epoch 7/20 | Train Loss: 0.7014 | Train Acc: 0.7535 | Test Loss: 0.8829 | Test Acc: 0.6999
Epoch 8/20 | Train Loss: 0.6448 | Train Acc: 0.7731 | Test Loss: 0.9040 | Test Acc: 0.6865
Epoch 9/20 | Train Loss: 0.5955 | Train Acc: 0.7909 | Test Loss: 0.9100 | Test Acc: 0.6944
Epoch 10/20 | Train Loss: 0.5441 | Train Acc: 0.8096 | Test Loss: 0.9304 | Test Acc: 0.6996
Epoch 11/20 | Train Loss: 0.5010 | Train Acc: 0.8241 | Test Loss: 0.9291 | Test Acc: 0.7042
Epoch 12/20 | Train Loss: 0.4564 | Train Acc: 0.8395 | Test Loss: 0.9815 | Test Acc: 0.6996
Epoch 13/20 | Train Loss: 0.4129 | Train Acc: 0.8561 | Test Loss: 1.0084 | Test Acc: 0.6964
Epoch 14/20 | Train Loss: 0.3772 | Train Acc: 0.8683 | Test Loss: 1.0568 | Test Acc: 0.7004
Epoch 15/20 | Train Loss: 0.3418 | Train Acc: 0.8790 | Test Loss: 1.0777 | Test Acc: 0.7018
Epoch 16/20 | Train Loss: 0.2995 | Train Acc: 0.8947 | Test Loss: 1.1803 | Test Acc: 0.7038
Epoch 17/20 | Train Loss: 0.2702 | Train Acc: 0.9047 | Test Loss: 1.2436 | Test Acc: 0.6936
Epoch 18/20 | Train Loss: 0.2385 | Train Acc: 0.9174 | Test Loss: 1.3116 | Test Acc: 0.6948
Epoch 19/20 | Train Loss: 0.2103 | Train Acc: 0.9277 | Test Loss: 1.4349 | Test Acc: 0.6959
Epoch 20/20 | Train Loss: 0.1922 | Train Acc: 0.9334 | Test Loss: 1.4932 | Test Acc: 0.6923
```

Базова CNN показала найкращий результат серед усіх підходів і досягла точності близько 70%. Це підтверджує, що для даної задачі класичні архітектури працюють ефективніше.

Висновок по лабаторній роботі

Було досліджено можливість використання моделі Mamba як backbone (основної частини моделі, що відповідає за витяг ознак) для задачі класифікації зображень. Реалізовано та протестовано кілька підходів, що дозволило порівняти їх ефективність у однакових умовах. Отримані результати показали, що використання замороженого backbone не дає прийнятної якості класифікації, оскільки модель не здатна адаптувати свої ознаки до нової задачі. Водночас донавчання всієї моделі дозволяє суттєво покращити результати, що свідчить про можливість ефективного використання Mamba після адаптації. Разом із тим, базова CNN-модель продемонструвала найкращі результати серед розглянутих підходів. Це підтверджує, що для задач класифікації зображень невеликого розміру класичні згорткові нейронні мережі залишаються більш придатними та ефективними.

Таким чином, можна зробити висновок, що моделі типу Mamba мають потенціал у задачах комп'ютерного зору, проте потребують донавчання і на даний момент поступаються спеціалізованим архітектурам у подібних задачах.